PYTHON ACTIVITY BASED LEARNING

PROBLEM-8:

Campus Event Seating and Entry Validation: Generate seating assignments from nested lists, validate tickets against registration identifiers, detect duplicate entries, and report available seating capacity.

Objective:

To develop a Campus Event Seating and Entry Validation System that generates seating assignments using nested lists, validates event tickets against registered student identifiers, detects duplicate entries, identifies invalid tickets, and calculates available seating capacity to ensure organized seating and secure event entry.

Algorithm:

1. Start.

2.Input the number of registered students and their registration IDs.

3.Input the number of tickets and each ticket ID with its registration ID.

4.Input the number of auditorium rows and seats per row.

5.Calculate total seating capacity as rows × columns.

6.Create a seating matrix and initialize all seats as EMPTY.

7.Initialize counters for valid entries, invalid tickets, duplicate entries, and full-auditorium rejections.

8.For each ticket:

- Check whether the registration ID exists in the registered list.
- If not registered, mark the ticket as invalid.
- If the registration ID has already entered, mark it as a duplicate.
- Otherwise, find the first available seat.
- If a seat is available, assign it and update the valid-entry count.
- If no seat is available, reject entry because the auditorium is full.

9.Display the seating arrangement.

10.Calculate available seats as total capacity minus occupied seats.

11.Display the event summary, including valid entries, invalid tickets, duplicate entries, full-auditorium rejections, occupied seats, and available seats.

12. Stop.

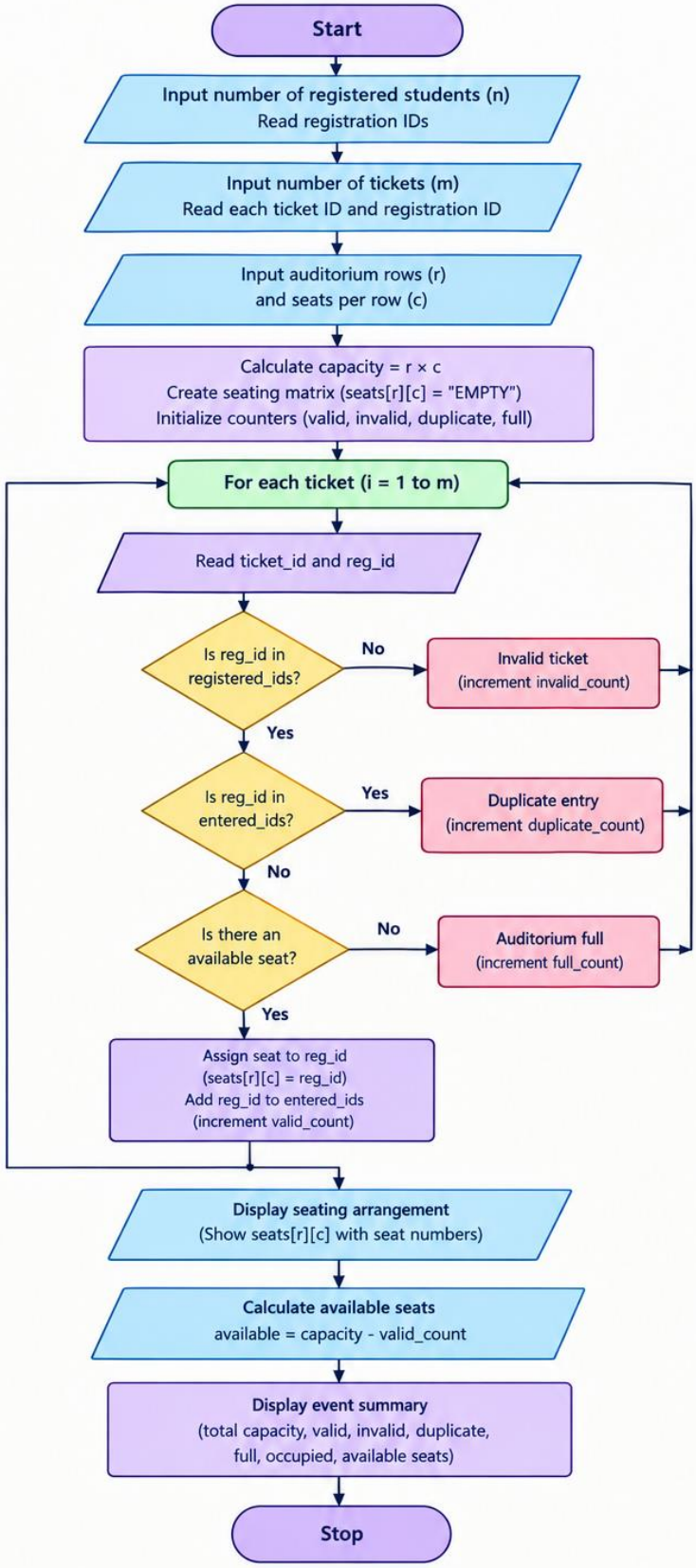
Pseudocode:

```
BEGIN
INPUT number_of_registered_students
INITIALIZE registered_ids = empty list
FOR each registered student
    INPUT registration_id
    ADD registration_id to registered_ids
END FOR
INPUT number_of_tickets
INITIALIZE tickets = empty list
FOR each ticket
    INPUT ticket_id, registration_id
    ADD ticket details to tickets
END FOR
INPUT rows, columns
capacity = rows * columns
INITIALIZE all seats as EMPTY
INITIALIZE entered_ids = empty list
INITIALIZE valid_count = 0
INITIALIZE invalid_count = 0
INITIALIZE duplicate_count = 0
INITIALIZE full_count = 0
FOR each ticket in tickets
    IF registration_id NOT IN registered_ids THEN
        DISPLAY INVALID
```

```
        invalid_count = invalid_count + 1
    ELSE IF registration_id IN entered_ids THEN
        DISPLAY DUPLICATE ENTRY
        duplicate_count = duplicate_count + 1
    ELSE
        FIND the first EMPTY seat
        IF an EMPTY seat exists THEN
            ASSIGN registration_id to that seat
            ADD registration_id to entered_ids
            valid_count = valid_count + 1
            DISPLAY VALID and seat number
        ELSE
            DISPLAY AUDITORIUM FULL
            full_count = full_count + 1
        END IF
    END IF
END FOR
DISPLAY seating arrangement
available = capacity - valid_count
DISPLAY total capacity
DISPLAY valid entries
DISPLAY invalid tickets
DISPLAY duplicate entries
DISPLAY full-auditorium rejections
DISPLAY occupied seats
DISPLAY available seats
STOP
```

Flowchart:

Campus Event Seating and Entry Validation (Flow Chart)



Input Details

- Number of registered students (n)
- Registration IDs (n)
- Number of tickets (m)
- Ticket ID and Registration ID (m)
- Auditorium rows (r)
- Seats per row (c)

Rules / Conditions

- A ticket is valid only if the registration ID is in the registered list.
- A student can occupy only one seat (duplicate entry check).
- Assign the first available seat in row-major order (Row 1 → Col 1, Row 1 → Col 2, ...).
- If no seat is available, entry is rejected (auditorium full).

Seating Matrix (Nested List)

- seats = [["EMPTY" for c in range(cols)] for r in range(rows)]
- Each seat stores the registration ID of the student.
- Display as: R1C1:ST101 | R1C2:ST102 | ...

Key Variables

- registered_ids – list of valid registrations
- entered_ids – list of students already entered
- valid_count – number of successful entries
- Invalid_count – invalid tickets
- duplicate_count – duplicate entries
- full_count – rejected due to full capacity
- capacity – total seats (rows × cols)
- available – remaining seats

Output

- Seating arrangement (matrix format)
- Valid entries with seat numbers
- Invalid tickets
- Duplicate entries
- Rejected (auditorium full)
- Event summary (capacity, counts, available seats)

Python Program:

```
# Campus Event Seating and Entry Validation

n = int(input("Enter number of registered students: "))
registered_ids = []

for i in range(n):
    reg_id = input("Enter registration ID: ")
    registered_ids.append(reg_id)

m = int(input("\nEnter number of tickets: "))
tickets = []

for i in range(m):
    print("\nTicket", i + 1)
    ticket_id = input("Enter ticket ID: ")
    reg_id = input("Enter registration ID: ")
    tickets.append((ticket_id, reg_id))

rows = int(input("\nEnter number of rows: "))
cols = int(input("Enter seats per row: "))

capacity = rows * cols

# Create seating matrix
seats = [["EMPTY" for c in range(cols)]
         for r in range(rows)]
```

```
entered_ids = []
valid_count = 0
invalid_count = 0
duplicate_count = 0
full_count = 0

print("\nCAMPUS EVENT ENTRY VALIDATION")
print("-" * 45)

for ticket_id, reg_id in tickets:
    print("\nTicket:", ticket_id, "Registration:", reg_id)

    if reg_id not in registered_ids:
        print("Status: INVALID - Not registered")
        invalid_count += 1

    elif reg_id in entered_ids:
        print("Status: DUPLICATE ENTRY")
        duplicate_count += 1

    else:
        assigned = False

        # Find the first available seat
        for r in range(rows):
            for c in range(cols):
                if seats[r][c] == "EMPTY" and not assigned:
                    seats[r][c] = reg_id
                    entered_ids.append(reg_id)
```

```
                valid_count += 1
                assigned = True

                print("Status: VALID")
                print("Seat: Row", r + 1,
                      "Column", c + 1)

        if not assigned:
            print("Status: REJECTED - Auditorium full")
            full_count += 1

# Display seating arrangement
print("\nSEATING ARRANGEMENT")
print("-" * 45)

for r in range(rows):
    for c in range(cols):
        print("R" + str(r + 1) + "C" + str(c + 1)
              + ":" + seats[r][c], end=" | ")
    print()

available = capacity - valid_count

print("\nEVENT SUMMARY")
print("-" * 45)
print("Total seating capacity:", capacity)
print("Valid entries:", valid_count)
print("Invalid tickets:", invalid_count)
print("Duplicate entries:", duplicate_count)
```

print("Rejected due to full auditorium:", full_count)

print("Occupied seats:", valid_count)

print("Available seats:", available)

Output Screen Shots:

